Speaker Notes

Quantum Algorithms — 1-Hour Lecture

Part I: Foundations (20 minutes)

Slide 1 Title Slide

1 min

Welcome everyone. Today we go from zero quantum computing knowledge to understanding the key algorithms that make quantum computing exciting — Grover's search, Shor's factoring, and modern NISQ-era techniques. We keep the math light and focus on intuition. Feel free to ask questions at any time.

Slide 2 Roadmap

1 min

Three parts: foundations (qubits, gates, entanglement), the big algorithms (Grover's, Shor's), and NISQ-era practical algorithms (Hamiltonians, VQE, QAOA, error mitigation). Each section builds on the previous one.

Slide 3 Why Quantum Computing?

3 min

Classical computers are incredibly powerful, but some problems take longer than the age of the universe. Quantum computing is not about being faster at everything — it exploits quantum physics to restructure certain computations. Three key resources: superposition, entanglement, interference. Emphasize: quantum computers are NOT universally faster — they are faster for specific structured problems.

Slide 4 Classical Bits vs. Qubits

4 min

This is the most important conceptual leap. A classical bit is definitively 0 or 1. A qubit is described by two complex amplitudes alpha and beta. The coin analogy is imperfect but useful. Stress that |alpha|^2 and |beta|^2 are probabilities of measuring 0 or 1. Dirac notation |0> and |1> is just labeling — do not let the notation intimidate them.

Slide 5 Quick Notation Guide

1 min

Reference slide — do not spend too long. Key point: |+> and |-> differ only by a minus sign (relative phase), which is the mechanism behind quantum interference and every quantum algorithm. Mention we will see why this minus sign matters with Grover's.

Slide 6 Quantum Gates

4 min

Three gates are enough for today. X gate = classical NOT. Hadamard (H) is the workhorse — puts a qubit into equal superposition. CNOT is the two-qubit gate that creates entanglement.

Walk through: apply H to |0> then CNOT with second qubit at |0> to get Bell state (|00>+|11>)/sqrt(2). All quantum gates are reversible (unitarity) — fundamental physics constraint.

Slide 7 **Entanglement** 3 min

In the Bell state, measuring the first qubit and getting 0 guarantees the second is 0. Einstein called this "spooky action at a distance." Clarify: does NOT allow faster-than-light communication (no-communication theorem). Entanglement is a resource that quantum algorithms consume.

Slide 8 **Deutsch–Jozsa Algorithm** 2 min

The "hello world" of quantum algorithms. Explain the problem: given a black-box function f that is either constant or balanced, determine which. Classically requires 2^(n-1)+1 queries in the worst case. Quantumly: one query suffices. Introduce the superposition-oracle-interference pattern that will recur in Grover's and Shor's.

Slide 9 **Deutsch–Jozsa: Circuit** 2 min

Walk through the circuit diagram. Input qubits start at |0>, ancilla at |1>. All get H gates. Then the oracle U_f is applied once, then H again on the input qubits, then measure. If f is constant you get |00>; if balanced, you get something else. The key insight: querying in superposition gives info about ALL inputs at once. Interference extracts a global property in one shot.

Part II: The Big Algorithms (20 minutes)

Slide 10 **Grover's Search — Problem** 2 min

Quadratic speedup for unstructured search. Provably optimal — cannot do better than sqrt(N). Concrete example: searching 1 million entries takes 500K lookups classically, 1000 with Grover's. Not as dramatic as Shor's exponential speedup, but applies to a much broader class of problems. Works for any "find the input satisfying this condition" problem.

Slide 11 **Grover's — How It Works** 2 min

Start with n qubits in |0>, apply H to get uniform superposition. Then repeat sqrt(N) times: (1) Oracle O_f flips the phase of the target state, (2) Diffusion operator reflects all amplitudes about the mean. After sqrt(N) repetitions, the target state has amplitude close to 1. Measure to find it.

Best teaching tool: draw a bar chart on the board. Show oracle dipping one bar below zero, then reflection about the mean boosting it above the others.

Slide 12 **Grover's — Circuit** 2 min

Point out the structure in the circuit: H gates create uniform superposition, then the Grover iteration block (oracle + diffusion) repeats sqrt(N) times, then measurement. The dashed box highlights one iteration.

Emphasize this is a simplified view — the actual oracle and diffusion operators depend on the specific problem.

Slide 13 Grover's — Amplitude Amplification Graph

2 min

The graph shows probability of finding the target vs. number of Grover iterations for N=64. Key points to emphasize:

- The probability oscillates sinusoidally — rises to nearly 1.0, then falls back down
- The peak is at approximately pi/4 times sqrt(N) = 6 iterations (marked with dashed line)
- Over-iterating decreases success — unlike classical search where more tries always helps
- This is uniquely quantum: the algorithm can "overshoot"
- Practical implication: you must know N (or estimate it) to know when to stop

Slide 14 Shor's — Motivation

2 min

This is why governments invest billions. RSA encryption relies on factoring being hard. Shor's breaks that assumption. We do not have a large enough quantum computer yet, but when we do, RSA is broken. Post-quantum cryptography: NIST standardized new algorithms in 2024. Key insight: factoring reduces to period finding, and quantum computers excel at finding periods.

Slide 15 Shor's — The Key Idea

2 min

Walk through the reduction: factoring N reduces to finding the period r of f(x) = a^x mod N. Once we know r, classical math gives us the factors via gcd(a^(r/2) +/- 1, N). Shor did not invent quantum factoring from scratch — he used a classical result (factoring = period finding) and showed QFT finds periods exponentially faster.

Slide 16 Shor's — Circuit

2 min

Walk through the circuit. Two registers: counting (top) and work (bottom). Apply H to the counting register to create superposition. Controlled-U_a computes a^x mod N in the work register. The inverse QFT on the counting register extracts the period. Measure the counting register and use continued fractions to find the period r. The QFT takes O(n^2) gates vs O(n times 2^n) classically.

Slide 17 QFT Intuition

2 min

Analogy: mystery signal goes through Fourier transform to reveal frequency. Similarly, f(x) = a^x mod N is periodic, QFT extracts the period. QFT operates on 2^n amplitude values using only n^2 operations. Catch: measurement gives one sample. But continued fractions extract the period from a few samples. Recurring theme: quantum gives clever sample, classical post-processing extracts answer.

Slide 18 Algorithm Comparison

2 min

All three follow the same meta-pattern: superposition, clever operation, interference. Speedup types: Deutsch-Jozsa exponential but artificial, Grover's quadratic but broadly applicable, Shor's super-polynomial with huge practical implications. Caveats: not universally faster, speedups need structure, Grover's is provably optimal.

Part III: The NISQ Era (20 minutes)

Slide 19 The NISQ Era 2 min

Transition. We discussed algorithms needing large, perfect quantum computers. Now: what can we do today? Current computers are noisy — every gate has error chance, qubits decohere quickly. Only short circuits work. NISQ approach: keep circuit shallow, classical computer handles optimization. But first we need to understand what a Hamiltonian is.

Slide 20 What is a Hamiltonian? 3 min

This is the key concept connecting physics to algorithms. A Hamiltonian H is a matrix encoding the total energy of a quantum system. Its eigenvalues are energy levels; eigenvectors are the corresponding states. The smallest eigenvalue is the ground state energy.

For chemistry: H encodes all electron-electron and electron-nucleus interactions. For optimization: H encodes the cost function — the ground state IS the optimal solution. For time evolution: H governs how quantum states change (Schrodinger equation).

Key challenge: for n qubits, H is a 2^n by 2^n matrix. Diagonalizing it classically is exponentially expensive. This is exactly the problem quantum computers can help with.

Slide 21 Pauli Decomposition 2 min

Any qubit Hamiltonian decomposes into a sum of Pauli strings (tensor products of I, X, Y, Z). The H2 example: a real molecule's energy operator becomes just 5 terms, each measurable.

Key insight: the number of Pauli terms grows polynomially (roughly n^4 for chemistry), not exponentially, even though the matrix is 2^n by 2^n. This makes measurement tractable.

Cannot measure H directly — must measure each term separately, average many shots, sum classically. This "term-by-term measurement" is the foundation of VQE.

Slide 22 The Variational Approach 2 min

Core idea behind all NISQ algorithms. Machine learning analogy: variational quantum circuit is like a neural network where "layers" are quantum gates and "weights" are rotation angles. Quantum computer evaluates the cost function, classical computer tunes parameters.

The variational principle guarantees any trial state gives an energy upper bound to the ground state. Minimizing E(theta) always approaches the true answer from above.

Slide 23 VQE: Variational Quantum Eigensolver 2 min

Explain the VQE loop: prepare trial state with parameterized circuit, measure Hamiltonian terms, compute energy estimate, feed back to classical optimizer, repeat. The ansatz design is a key challenge — it must be expressive enough to capture the physics but shallow enough to survive noise. Common choices: hardware-efficient ansatz, UCCSD ansatz. Current demonstrations: H_2, LiH, BeH_2. The challenge: scaling to industrially relevant molecules.

Slide 24 VQE: Ansatz Circuit
1 min

Walk through the ansatz circuit diagram. Two qubits start at |0>. Ry(theta) gates apply parameterized rotations. CNOT gates create entanglement. The pattern repeats in layers — more layers mean more expressiveness but also more noise. After the circuit, measure in the Pauli bases needed for each Hamiltonian term. Repeat many times to get expectation values. Classical optimizer updates theta values.

Slide 25 QAOA: Quantum Approximate Optimization
2 min

Walk through the QAOA idea. All qubits start in |+> (uniform superposition via H gates). The problem unitary exp(-i gamma H_C) encodes the graph structure — for Max-Cut, this is ZZ interactions between connected vertices. The mixer Rx(beta) enables amplitude redistribution.

One layer = one pair of (problem, mixer). More layers p improve the approximation ratio but deepen the circuit. The gamma and beta angles are optimized classically.

Be honest: open question whether QAOA beats classical heuristics for practical problems.

Slide 26 QAOA: Circuit
1 min

Point out the structure: H gates for initial superposition, then the problem unitary (ZZ gates between connected qubits with parameter gamma) and mixer (Rx gates with parameter beta) form one QAOA layer. The dashed box highlights a single layer. More layers improve quality at the cost of circuit depth. After all layers, measure in the computational basis to read out the solution.

Slide 27 Error Mitigation I: ZNE
2 min

Zero-Noise Extrapolation. Run the same circuit at artificially increased noise levels. The graph on the slide shows the key idea: measured points at noise scales lambda = 1, 2, 3 and the extrapolation to lambda = 0 (the ideal, noiseless answer).

How to increase noise: insert pairs of CNOT gates that cancel logically but accumulate noise (gate folding), or stretch pulse durations. Then fit a curve (linear, exponential, polynomial) through the noisy data points and extrapolate to zero.

Pros: simple, no detailed noise model needed. Cons: assumes smooth noise scaling, which may not hold for complex circuits. Polynomial cost in additional shots.

Slide 28 Error Mitigation II: PEC
2 min

Probabilistic Error Cancellation. First characterize each gate's noise channel via tomography. Then express the ideal gate as a linear combination of noisy operations. Sample circuits from this combination with signed weights. The weighted average converges to the noiseless result.

Key insight: this is exact in principle, unlike ZNE. The catch: sampling overhead scales exponentially with circuit depth (gamma^(2d)), making it practical only for shallow circuits. Fortunately, that is exactly what NISQ devices run.

Sometimes called "quasi-probability" method. Related to the concept of negative probabilities in quantum mechanics.

Slide 29 Error Mitigation III: Measurement 2 min

The simplest technique. Prepare known basis states, measure them many times, build a confusion matrix M. Then apply M^(-1) to correct measured distributions. The example 4x4 matrix shows 94-95% readout fidelity (typical for current hardware).

Only fixes readout errors, not gate errors — but readout errors are often the dominant source. For large n, the full 2^n by 2^n matrix is intractable, so tensor product approximations are used (assume qubit readout errors are independent).

This is almost always worth doing — simple and effective. Most quantum computing frameworks (Qiskit, Cirq) have built-in support for it.

Slide 30 NISQ Landscape 2 min

Snapshot of where the field is. Numbers change quickly — emphasize order of magnitude. 2023 milestone: Google showed adding more physical qubits to an error-correcting code actually reduces logical error rate (crossing threshold theorem). Trapped ions: higher fidelity, slower. Superconducting: faster, noisier. Neutral atoms: newer, promising scalability. Update qubit counts based on latest news.

Wrap-Up (5 minutes)

Slide 31 Summary 2 min

Recap three layers: physical foundations, theoretical algorithms, practical NISQ approaches. Key takeaway: quantum speedups are real but specific — require mathematical structure. Quantum computing is not magic; different computational model excelling at certain tasks. We are at an exciting inflection point where theory meets hardware. Emphasize: Hamiltonians bridge physics and algorithms, error mitigation lets us extract useful results from imperfect hardware today.

Slide 32 Thank You / Q&A 3 min

End with Feynman quote. Open floor for questions. Common ones:

(1) "When will quantum computers break RSA?" — probably 10-20+ years, but migration to post-quantum crypto should start now.

(2) "Can I use a quantum computer today?" — yes! IBM offers free access via Qiskit. Google's Cirq and Xanadu's PennyLane are also great.

(3) "Will quantum computers replace classical ones?" — no, they are complementary. Quantum is a specialized accelerator, not a general replacement.

(4) “What about error mitigation vs error correction?” — mitigation is a stopgap for today; correction is the long-term solution. They serve different purposes and will coexist.